CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 9: State persistence in multi-agent pipelines

> **Exam objective:** Design state persistence strategies for multi-agent pipelines that enable reliable resumption after interruption without repeating completed work or losing prior findings.

Q1.

A long-running analysis pipeline has four phases: discovery, classification, remediation proposal, and report generation. The process may be interrupted between phases. Which persistence design best supports reliable resume?

A. Keep all state only in coordinator memory until the very end B. Write durable checkpoints after each completed phase, including outputs and next-step metadata C. Recompute every prior phase on restart to guarantee freshness D. Persist only the final report because intermediate data is transient

Answer: B **Explanation:** Phase checkpoints let the system restart from the last known-good boundary without replaying all prior work. Holding state only in memory or persisting only the final report makes interruption expensive and lossy.

Q2.

A team is deciding between storing a single mutable `current_status` record and keeping an append-only event log plus periodic snapshots. They expect frequent interruptions and want strong auditability. Which option is better?

A. Single mutable status record only B. Append-only event log with periodic snapshots C. No persistence until the workflow completes successfully D. Randomly sampled logs for storage efficiency

Answer: B **Explanation:** Event logs preserve the history of what happened, while snapshots speed restoration. A single mutable record loses causality and makes forensic reconstruction much harder.

Q3.

Multiple worker agents process independent tasks in parallel. After a crash, the coordinator must know exactly which tasks finished, which are in progress, and which were never started. What should be persisted?

A. Only the total number of tasks completed B. Per-task durable status, result references, and idempotency identifiers C. Only a final merged summary once all workers finish D. A copy of the original user prompt and nothing else

Answer: B **Explanation:** Reliable resumption in parallel systems requires item-level state, not aggregate counts. Idempotency identifiers help avoid duplicate completion or side effects when tasks are retried.

Q4.

A resumed pipeline must avoid rerunning expensive static analysis on modules already processed successfully. Which mechanism most directly supports that goal?

A. Deleting all prior results on startup to ensure consistency B. Persisting terminal task states and reusable result references for completed modules C. Forcing all workers to start from the first module every time D. Reducing log verbosity so restarts happen faster

Answer: B **Explanation:** If completed work is durably recorded with stable result references, the restart logic can skip it safely. Restarting from scratch wastes time and does not improve correctness.

Q5.

A worker reads paginated API results from an external service. If interrupted, it should continue from the last processed page without duplicating records. What is the best state to persist?

A. Only the first page of the response B. The next-page cursor or token plus normalized records already committed C. A screenshot of the API documentation D. The worker's chain-of-thought text only

Answer: B **Explanation:** Cursor-based progress and committed result state allow precise continuation after interruption. Documentation and reasoning text do not tell the system where to resume safely.

Q6.

A human approval step pauses a remediation workflow after the fix proposal is generated. What persistence design best supports later resumption?

A. Store the workflow as `awaiting approval` along with the proposed fix artifacts and decision context B. Discard the proposal and regenerate it after approval arrives C. Auto-approve on restart so execution can continue D. Persist only the approval request timestamp

Answer: A **Explanation:** Human-gated workflows need durable state that captures both the pause reason and the materials needed to continue. Persisting only a timestamp loses the actual work product and decision basis.

Q7.

A multi-agent research system aggregates evidence from several specialist workers. To support resumption and traceability, what should be stored durably?

A. Only the final conclusion paragraph B. Shared evidence artifacts, citations, and each worker's contribution metadata C. Nothing, because evidence can always be recollected later D. Only the coordinator's last message to each worker

Answer: B **Explanation:** In research pipelines, the intermediate evidence is often as important as the conclusion because later stages depend on it. Recollecting it may be costly, inconsistent, or impossible if sources change.

Q8.

A worker writes a JSON result file and then marks its task complete in the state store. A crash occurs after the completion flag is written but before the file is fully flushed. Which improvement is best?

A. Mark completion before writing to reduce latency B. Use atomic write-and-commit ordering so the task is marked complete only after the artifact is durably stored C. Remove the completion flag and infer success from optimism D. Retry the same task forever without any state checks

Answer: B **Explanation:** Completion should reflect durable reality, not intended state. Atomic or ordered commits prevent the system from resuming under the false assumption that the output already exists.

Q9.

A sequential pipeline passes structured outputs from one phase to the next. Over time, the schema of those outputs evolves. What persistence practice best protects resumability?

A. Never version intermediate data schemas B. Persist versioned state so resumed phases can validate or migrate prior outputs C. Convert all structured state into an untyped text blob D. Re-run the entire pipeline whenever any field changes

Answer: B **Explanation:** Versioned intermediate state lets the system detect incompatibilities and migrate safely instead of silently misreading old data. Untyped blobs make reliable validation harder, not easier.

Q10.

A pipeline resumes after a codebase change that invalidates earlier findings. Which persistence strategy best balances reuse and correctness?

A. Reuse all prior results unconditionally B. Store input fingerprints and invalidation rules so stale checkpoints can be detected and recomputed selectively C. Delete the entire state store whenever a restart occurs D. Ignore the codebase change if the previous run was more than 50% complete

Answer: B **Explanation:** Checkpoints are only useful if the system can determine whether they still correspond to current inputs. Fingerprints and invalidation rules enable selective recomputation instead of either blind reuse or total discard.

Q11.

A rate-limited research job spans multiple external sources and may pause for hours between retries. What persisted data is most valuable for accurate resume?

A. Source-specific cursors, retry metadata, and already harvested artifact references B. Only a human-readable executive summary C. The number of prompts issued so far D. A screenshot of the dashboard showing rate limits

Answer: A **Explanation:** Durable source progress and retry metadata let the job resume efficiently without re-harvesting or violating limits unnecessarily. Summaries and screenshots do not encode actionable restart state.

Q12.

A pipeline performs side effects such as posting tickets and sending notifications. After restart, it must not create duplicates. Which design is best?

A. Avoid storing any side-effect metadata so the pipeline stays simple B. Persist an idempotent operation log or deduplication keys for each external action C. Re-run all side effects because duplicates are harmless D. Disable retries on any failed external action

Answer: B **Explanation:** Exactly-once behavior across restarts depends on durable deduplication of side effects. Without that record, retries can easily create duplicate tickets, emails, or chat posts.

Q13.

Which statement best distinguishes what should be persisted versus what can remain ephemeral in a multi-agent pipeline?

A. Persist every token of internal reasoning to maximize reproducibility B. Persist task state, decisions, artifacts, and references; keep transient scratch reasoning ephemeral unless explicitly needed C. Persist nothing except timestamps because storage is expensive D. Keep artifacts ephemeral but persist only free-form summaries

Answer: B **Explanation:** Durable state should focus on the information needed to resume, audit, and consume downstream. Persisting every internal thought is rarely necessary and increases cost and exposure without proportionate benefit.

Q14.

A coordinator assigned 100 tasks to workers before crashing. On restart, it must reclaim tasks from workers that died and avoid duplicating work already underway elsewhere. Which persistence pattern is best?

A. Durable task queue entries with leases or heartbeats B. Stateless round-robin assignment with no coordination record C. A single text file saying "workers were busy" D. Immediate reassignment of every task on startup regardless of worker status

Answer: A **Explanation:** Lease- or heartbeat-based task ownership lets the coordinator distinguish abandoned work from healthy in-progress work. Blind reassignment causes duplication, while stateless scheduling loses operational truth.

Q15.

What is the strongest validation test for a pipeline designed to resume without repeating completed work?

A. Measure only the average runtime on successful uninterrupted runs B. Intentionally interrupt the pipeline mid-execution and verify restart resumes from persisted checkpoints without duplicate side effects C. Confirm the final report looks correct when no failures occur D. Disable persistence and compare output length across runs

Answer: B **Explanation:** Resumability should be tested under actual interruption conditions, not inferred from happy-path behavior. The key proof is that completed work is preserved and side effects are not duplicated after recovery.